N-gram language models

Class 3 of 24: Natural Language Processing

CSE 4811 / DS 4213 (Fall 2026)

Muhammad Anwarul Azim
Lecturer, Dept. of CSE
United International University

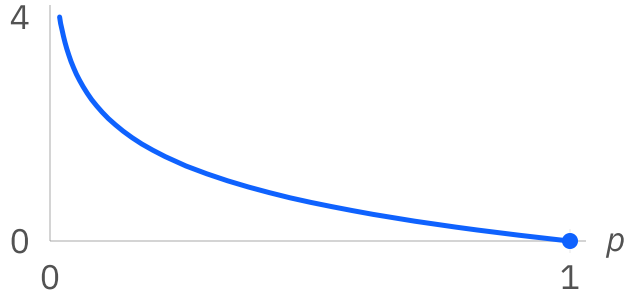
Recap

Naive Bayes, a model for classification

$$p(k \mid \text{text}) = \frac{p(\text{text} \mid k)\, p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$$

The loss

$$L(p) = -\log p$$



Naive Bayes can write

match	wins	star	match	wins	song	match	film	wins	match

Each row of its table can also write text, one word at a time.

Word probabilities

Probabilities	match	star	film	wins	song	Total
sports	0.40	0.20	0.05	0.30	0.05	**1**
entertainment	0.05	0.30	0.35	0.10	0.20	**1**

$$p(w \mid k) = \frac{\text{count}(w, k)}{\text{words in topic } k}$$

Dividing each count by its row total gives $p(w \mid k)$. Each row adds up to 1.

Classifying a headline

Test headline `Star film wins`

Probabilities	match	star	film	wins	song
sports	0.40	0.20	0.05	0.30	0.05
entertainment	0.05	0.30	0.35	0.10	0.20

Topic	Likelihood	Prior	Likelihood × prior	Posterior
sports	0.20 × 0.05 × 0.30 = 0.003	0.6	0.0018	0.30
entertainment	0.30 × 0.35 × 0.10 = 0.0105	0.4	0.0042	**0.70**
Sum		**1**	**0.0060**	**1**

Posterior



Prediction: **entertainment**

The topic with the largest posterior is the prediction: entertainment, with 0.70.

How good was Naive Bayes?

Dev macro-F1, higher is better



Naive Bayes reaches 0.836. It classifies 666 articles wrongly, against 973.

What is missing?

1. Word order

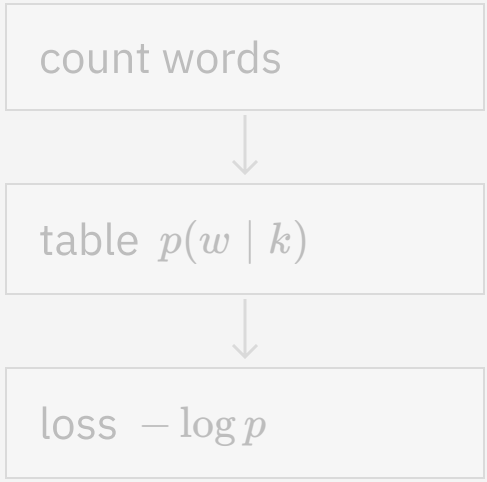
First

`Star film wins` ⟶ `film` `star` `wins`

`Wins film star` ⟶ `film` `star` `wins`

the same bag

2. Learning from the loss



no way back: the loss cannot change the counts

We first address word order.

Part 1: Word order

Each word, given the words before it.

How can we add order?

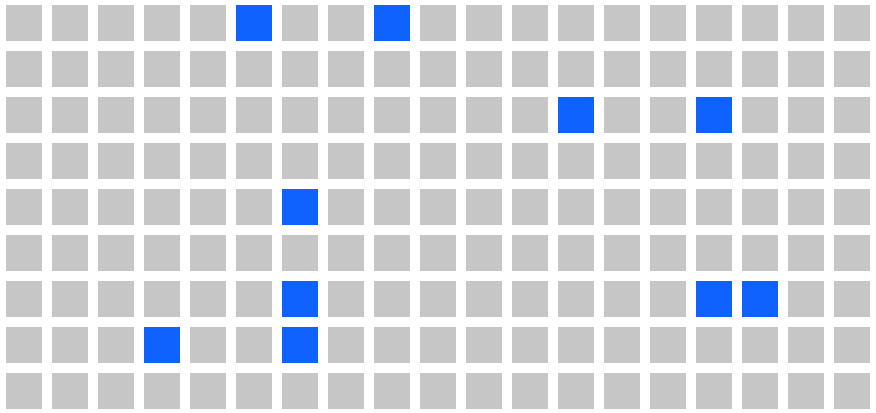
A dev headline `Soccer Star George Weah Wins Liberian Presidential Election`

soccer	$p(\text{soccer})$
soccer star	$p(\text{star} \mid \text{soccer})$
soccer star george	$p(\text{george} \mid \text{soccer star})$
soccer star george weah	$p(\text{weah} \mid \text{soccer star george})$
soccer star george weah wins	$p(\text{wins} \mid \text{soccer star george weah})$

The chain rule $p(w_1 \dots w_n) = \prod_{i=1}^{n} p(w_i \mid w_1 \dots w_{i-1})$

The chain rule of probability writes the probability of a text exactly as this product.

Conditional probability



`soccer`: 171 times

followed by `star`: 10 times

$$p(\text{star} \mid \text{soccer}) = \frac{\text{count}(\text{soccer star})}{\text{count}(\text{soccer})} = \frac{10}{171} = 0.058$$

$$p(\text{star}) = \frac{\text{count}(\text{star})}{\text{all words}} = \frac{489}{1{,}811{,}637} = 0.00027$$

Without the word before it, star is 489 of the 1,811,637 words: a probability of 0.00027.

Can we count every history?

History	Count in the training texts
`soccer`	171
`soccer star`	10
`soccer star george`	**0**
`soccer star george weah`	**0**
`soccer star george weah wins`	**0**

$$p(\text{george} \mid \text{soccer star}) = \frac{0}{10} = 0 \qquad p(\text{weah} \mid \text{soccer star george}) = \frac{0}{0}$$

Problem: most long histories occur rarely or never in the training texts, so their probabilities are 0 or 0/0.

The Markov assumption

soccer star george weah wins liberian **presidential** **election**

count 185

$$p(\text{election} \mid \text{soccer star george weah wins liberian presidential})$$

$$\approx p(\text{election} \mid \text{presidential})$$

$$p(w_i \mid w_1 \dots w_{i-1}) \approx p(w_i \mid w_{i-N+1} \dots w_{i-1})$$

$$p(\text{election} \mid \text{presidential}) = \frac{\text{count}(\text{presidential election})}{\text{count}(\text{presidential})} = \frac{31}{185} = 0.17$$

Solved: a short history occurs often enough in the training texts to be counted.

Bigrams, trigrams and so on

Unigrams 1 word	`soccer` `star` `george` `weah` `wins`
Bigrams 2 words	`soccer star` `star george` `george weah` `weah wins`
Trigrams 3 words	`soccer star george` `star george weah` `george weah wins`
4-grams 4 words	`soccer star george weah` `star george weah wins`
5-gram 5 words	`soccer star george weah wins`

A text of five words has five unigrams, four bigrams, three trigrams, two 4-grams and one 5-gram.

Part 2: The bigram model

One table per topic: each word, given the word before it.

Counting pairs

<s> film wins award <s> film star wins award <s> star wins film award <s>

- entertainment

previous word \ next word	<s>	team	wins	match	star	award	film	**Total**
<s>	0	0	0	0	1	0	2	**3**
team	0	0	0	0	0	0	0	**0**
wins	0	0	0	0	0	2	1	**3**
match	0	0	0	0	0	0	0	**0**
star	0	0	2	0	0	0	0	**2**
award	3	0	0	0	0	0	0	**3**
film	0	0	1	0	1	1	0	**3**

The entertainment sentences are counted into a second table.

Counts to probabilities

■ sports

	<s>	team	wins	match	star	award	film
<s>	0	2/3	0	0	1/3	0	0
team	0	0	1/2	0	1/2	0	0
wins	0	0	0	2/3	0	1/3	0
match	2/2	0	0	0	0	0	0
star	0	0	2/2	0	0	0	0
award	1/1	0	0	0	0	0	0
film	–	–	–	–	–	–	–

■ entertainment

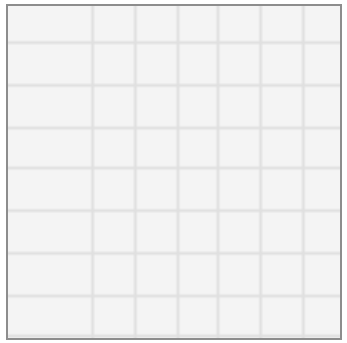
	<s>	team	wins	match	star	award	film
<s>	0	0	0	0	1/3	0	2/3
team	–	–	–	–	–	–	–
wins	0	0	0	0	0	2/3	1/3
match	–	–	–	–	–	–	–
star	0	0	2/2	0	0	0	0
award	3/3	0	0	0	0	0	0
film	0	0	1/3	0	1/3	1/3	0

$$p(w \mid v, k) = \frac{\text{count}(v\, w, k)}{\text{count}(v, k)}$$

A word that a topic does not use has an empty row: film in sports, team and match in entertainment.

Six topics, six tables

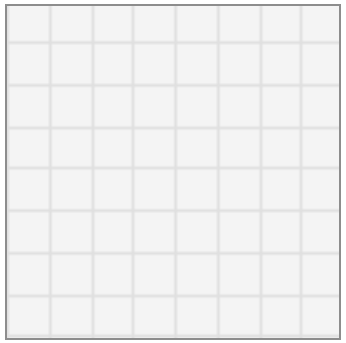
Each table: 110,609 rows × 110,609 columns ≈ 1.2×10^{10} cells



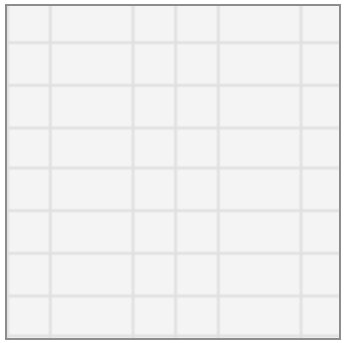
■ international



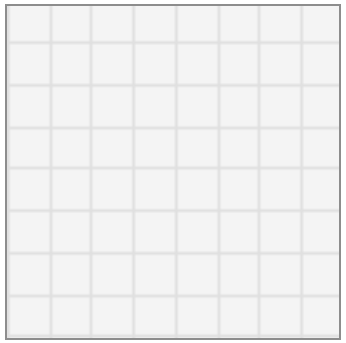
■ national politics



■ business and economy



■ entertainment



■ sports



■ science and technology

Different pairs in the training articles of each topic

205,294	186,068	202,177	202,436	198,696	189,218

The training articles of a topic contain about 200,000 different pairs. Only those cells are not 0.

The likelihood

Test headline `Star wins award`

`<s> star` | `star wins` | `wins award` | `award <s>`

■ sports

	<s>	team	wins	match	star	award	film
<s>	0	2/3	0	0	1/3	0	0
team	0	0	1/2	0	1/2	0	0
wins	0	0	0	2/3	0	1/3	0
match	2/2	0	0	0	0	0	0
star	0	0	2/2	0	0	0	0
award	1/1	0	0	0	0	0	0
film	–	–	–	–	–	–	–

■ entertainment

	<s>	team	wins	match	star	award	film
<s>	0	0	0	0	1/3	0	2/3
team	–	–	–	–	–	–	–
wins	0	0	0	0	0	2/3	1/3
match	–	–	–	–	–	–	–
star	0	0	2/2	0	0	0	0
award	3/3	0	0	0	0	0	0
film	0	0	1/3	0	1/3	1/3	0

$$p(\text{text} \mid k) = \prod_i p(w_i \mid w_{i-1}, k)$$

- $p(\text{text} \mid \text{sports}) = 1/3 \times 2/2 \times 1/3 \times 1/1 = 1/9 \approx 0.11$
- $p(\text{text} \mid \text{entertainment}) = 1/3 \times 2/2 \times 2/3 \times 3/3 = 2/9 \approx 0.22$

Entertainment is twice as likely as sports to write this headline.

The posterior

Test headline `Star wins award`

Topic	Likelihood	Prior	Likelihood × prior	Posterior
sports	1/9	0.5	0.0556	0.0556 / 0.1667 = **0.33**
entertainment	2/9	0.5	0.1111	0.1111 / 0.1667 = **0.67**
Sum		**1**	**0.1667**	**1**

Posterior



Divide each product by the sum. The bigram model predicts entertainment, the true topic.

A dev article

> A dev article, true topic: international
>
> `Soccer Star George Weah Wins Liberian Presidential Election His election marks the country's first democratic transition in over 70 years.`

Single words

- national politics **0.584** wrong

Pairs of words

- international **1.000** right

Word or pair	in international	in national politics
presidential	44	111
democratic	22	135
presidential election	20	6
democratic transition	0	0

As a pair, presidential election is more common in international, and democratic transition occurs in neither topic.

A misclassified article

> A dev article, true topic: business and economy
>
> `Markets Tumble Amid Brexit Chaos The vote to leave the EU has sparked a panic among investors around the world.`

Single words

- international **0.943** wrong

Pairs of words

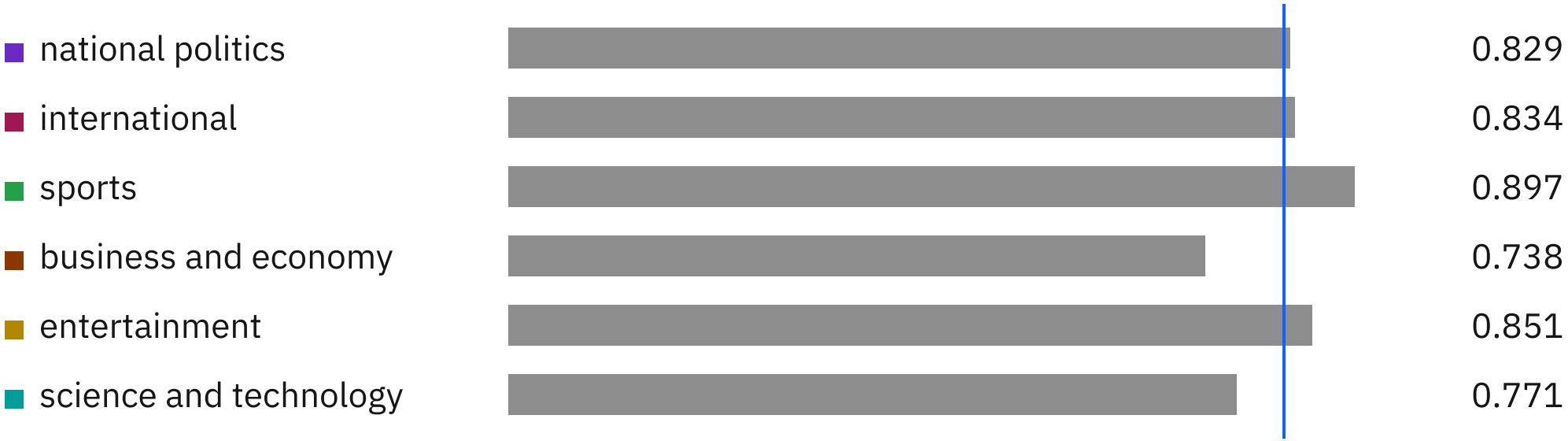
- international **0.999** wrong

Word or pair	in international	in business and economy
investors	1	53
markets	7	37
among investors	0	0
the eu	25	2
to leave	24	2
the vote	7	0

How good is the bigram model on the whole dev set?

Bigrams on the dev set

F1 for each topic



0.820 macro-F1, the mean of the six

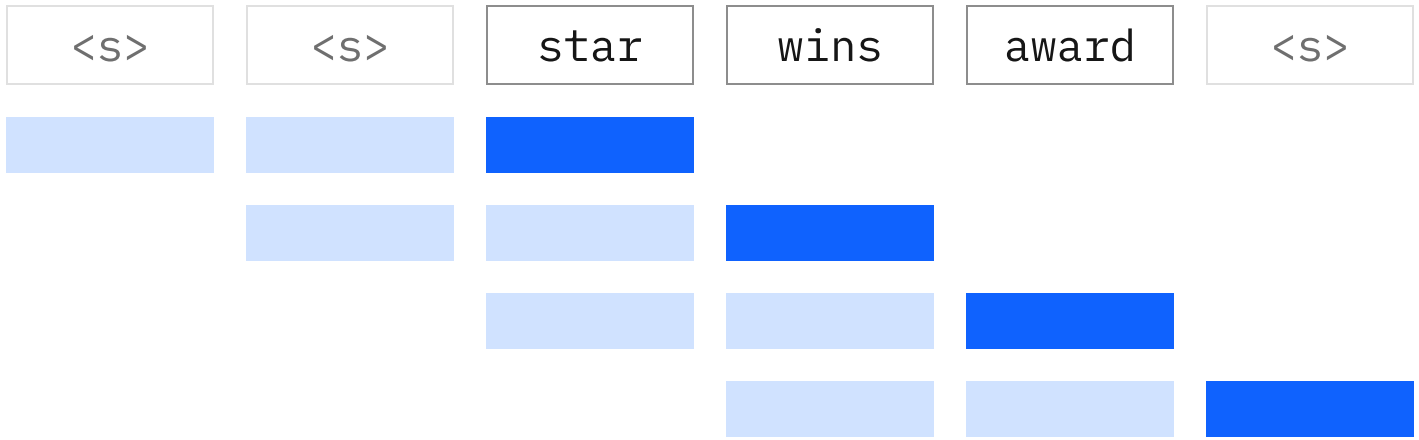
730 of 4,000 dev articles classified wrongly

Would a longer history help?

Part 3: More context

Two words of history for each word.

The trigram model



$$p(\text{star} \mid \text{<s> <s>})$$
$$p(\text{wins} \mid \text{<s> star})$$
$$p(\text{award} \mid \text{star wins})$$
$$p(\text{<s>} \mid \text{wins award})$$

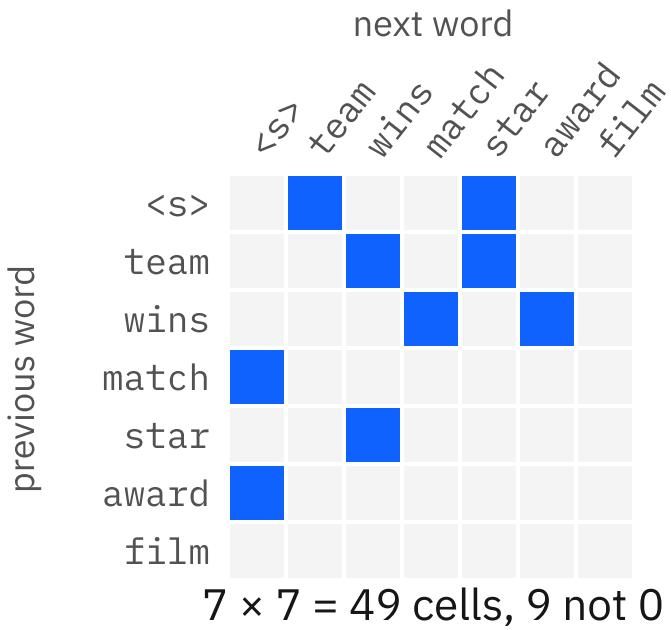
$$p(\text{text} \mid k) = \prod_i p(w_i \mid w_{i-2}\, w_{i-1}, k)$$

$$p(w \mid u\, v, k) = \frac{\text{count}(u\, v\, w, k)}{\text{count}(u\, v, k)}$$

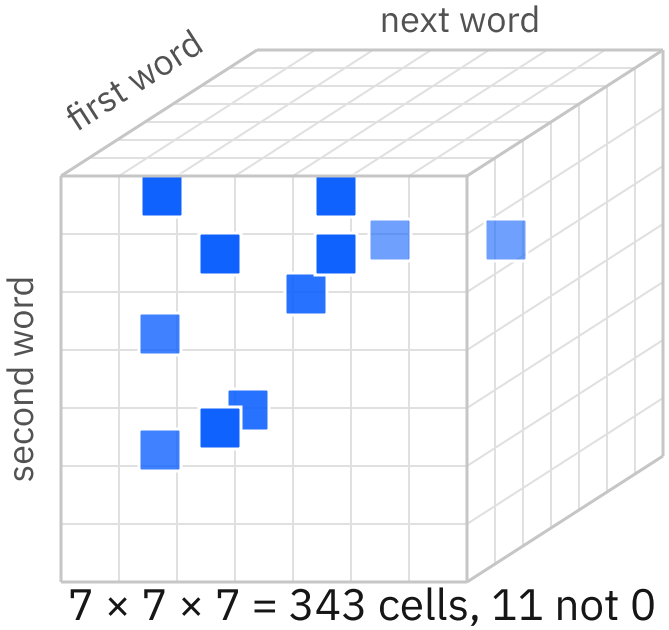
prior × likelihood → Bayes' formula → prediction

The prior, Bayes' formula and the prediction stay as before.

From a table to a cube



7 × 7 = 49 cells, 9 not 0



7 × 7 × 7 = 343 cells, 11 not 0

C1 data, each topic	Cells	Cells not 0	Cells not 0, about
Word list	110,608	33,313 to 37,309	1 in 3
Pair table	$110{,}609^2 = 1.2 \times 10^{10}$	186,068 to 205,294	1 in 60,000
Cube	$110{,}609^3 = 1.4 \times 10^{15}$	257,585 to 290,042	1 in 5 billion

Problem: almost every cell of the cube is 0. Of the dev trigrams, 84.4% never occur in the training texts of their topic.

Three models in code

Unigram

```
for words, k in zip(train_words, y):
    for w in words:
        count[k][w] += 1
        total[k] += 1
p = (count[k][w] + 1) / (total[k] + V)
```

Bigram

```
for words, k in zip(train_words, y):
    words = ["<s>"] + words + ["<s>"]
    for v, w in zip(words, words[1:]):
        count[k][v, w] += 1
        total[k][v] += 1
p = (count[k][v, w] + 1) / (total[k][v] + V)
```

Trigram

```
for words, k in zip(train_words, y):
    words = ["<s>", "<s>"] + words + ["<s>"]
    for u, v, w in zip(words, words[1:], words[2:]):
        count[k][u, v, w] += 1
        total[k][u, v] += 1
p = (count[k][u, v, w] + 1) / (total[k][u, v] + V)
```

All three add 1 to every count and V to every total, so an n-gram never seen still gets a small probability.

Results

Dev macro-F1, higher is better



Why does a longer history give a lower score?

How rare is a trigram?

> Markets Tumble Amid Brexit Chaos The vote to leave the EU has sparked a panic among investors around the world.

Count in the 40,000 training texts

Words	sparked 30	a 12,863	panic 18
Pairs	sparked a 5		
		a panic 1	
Trigram	sparked a panic 0		

The trigram itself never occurs.

The most common n-grams

Words		Pairs		Trigrams	
`the`	29,962	`of the`	2,735	`the u s`	358
`to`	15,877	`in the`	2,226	`one of the`	245
ও	12,976	করা হয়েছে	1,412	এ কথা বলেন	215
`of`	12,958	`it s`	1,213	বলে মন্তব্য করেছেন	199
`a`	12,863	`u s`	1,156	এসব কথা বলেন	186
Different words	**110,608**	Different pairs	**964,642**	Different trigrams	**1,496,818**
Seen only once	**45.4%**	Seen only once	**79.3%**	Seen only once	**92.9%**

Of all the different trigrams, 92.9% occur only once.

Unseen n-grams in an article

never occurs in the training texts

Words

markets | tumble | amid | brexit | chaos | the | vote | to | leave | the | eu | has | sparked | a | panic | among | investors | around | the | world

Pairs

markets tumble | tumble amid | amid brexit | brexit chaos | chaos the | the vote | vote to | to leave | leave the | the eu | eu has | has sparked | sparked a | a panic | panic among | among investors | investors around | around the | the world

Trigrams

markets tumble amid | tumble amid brexit | amid brexit chaos | brexit chaos the | chaos the vote | the vote to | vote to leave | to leave the | leave the eu | the eu has | eu has sparked | has sparked a | sparked a panic | a panic among | panic among investors | among investors around | investors around the | around the world

Share of all dev n-grams never seen in the training texts of their topic

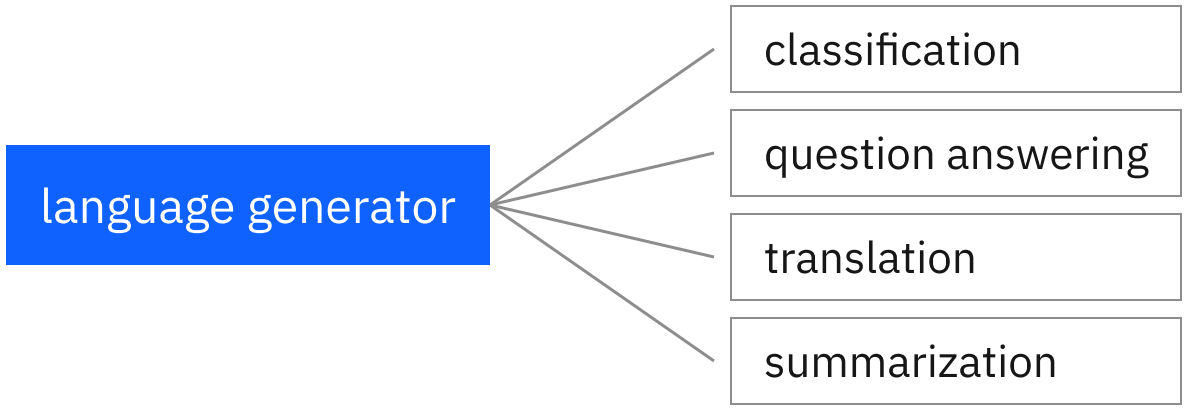
Words 3.5% Pairs 53.0% Trigrams 84.4%

Problem: an n-gram that never occurs gets almost the same probability in every topic, so it does not help to choose the topic.

Part 4: Generation

Writing text, and measuring a model on new text.

Generation



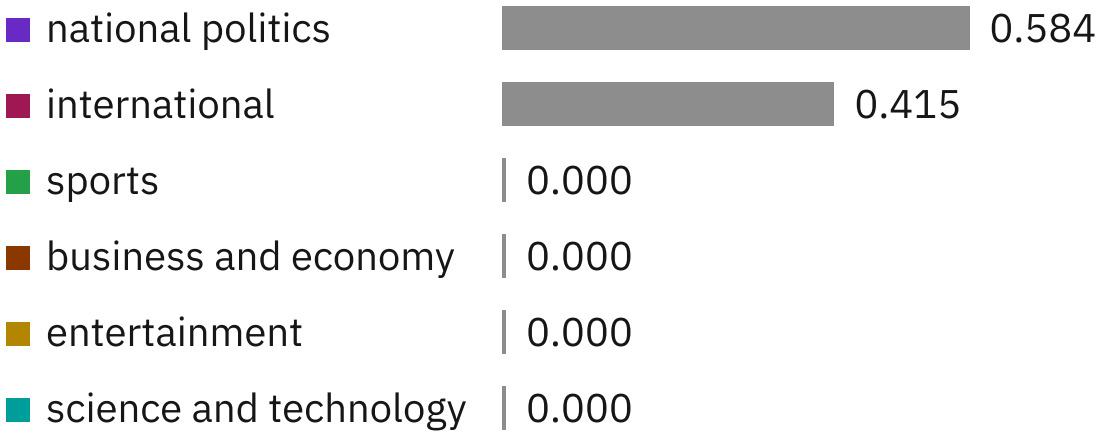
Unigram	$p(w \mid k)$
Bigram	$p(w \mid v, k)$
Trigram	$p(w \mid u\, v, k)$
A large language model	$p(\text{next token} \mid \text{all the words so far})$

Generation is the more important task, and generation needs word order.

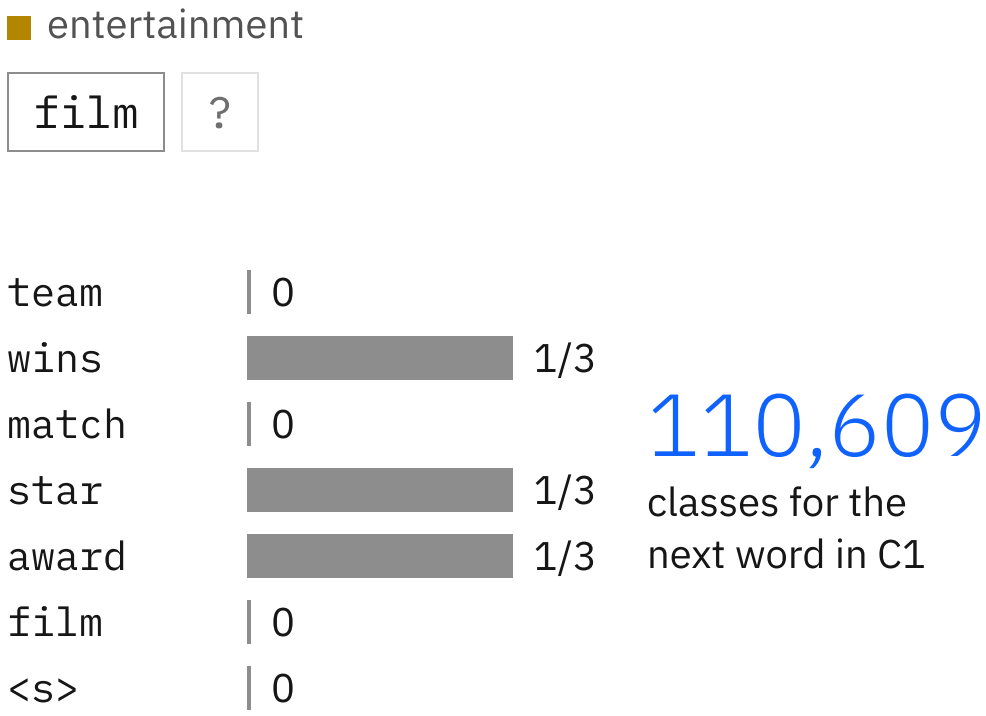
Choosing the next word

Classification

> Soccer Star George Weah Wins Liberian Presidential Election His election marks the country's first democratic transition in over 70 years.



Generation



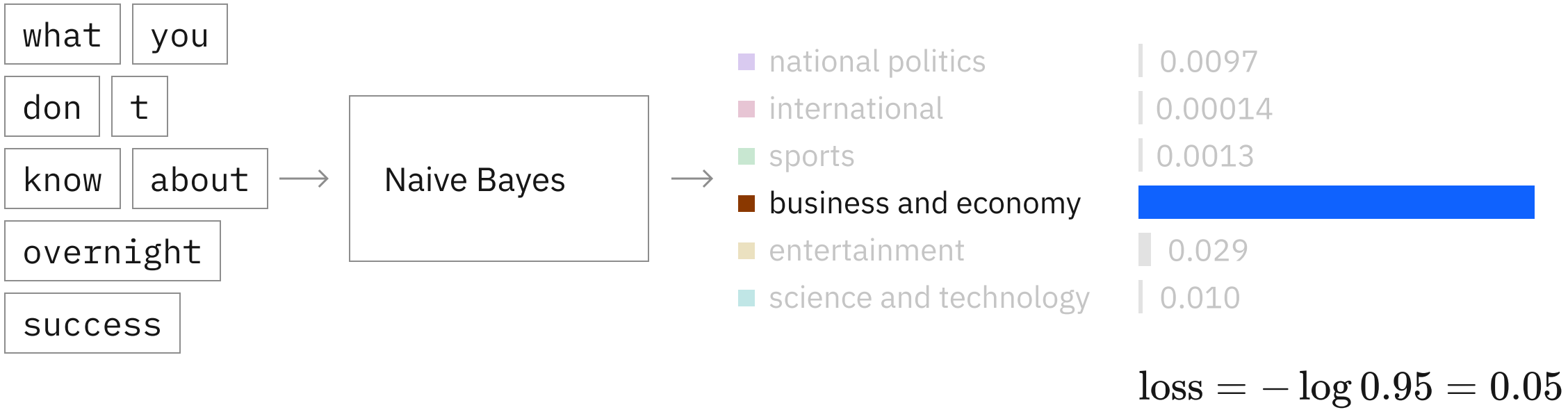
Generation is classification of the next word, with one class for each word.

Three models write

Topic	Unigram	Bigram	Trigram
business and economy	colts সাংবাদিকদের দিলেন plays অবশেষে a s happiness wants ২২ babies owned কিংবদন্তী ইউরোপিয়ান স্থানীয় পারছে বিভাগীয় the can a	year s olympic gold medal win over here </s>	টস জিতে ব্যাটিং নেয়া বাংলাদেশ শুরুতেই বিপর্যয়ে পড়ে রানের খাতা খুলার আগেই ফিরে গেছেন অভিষিক্ত সাইফ হাসান এবার ইনজুরিতে পড়লেন
entertainment			
international			
national politics			
science and technology			
sports			

Each model writes from the counts of the chosen topic, without adding 1.

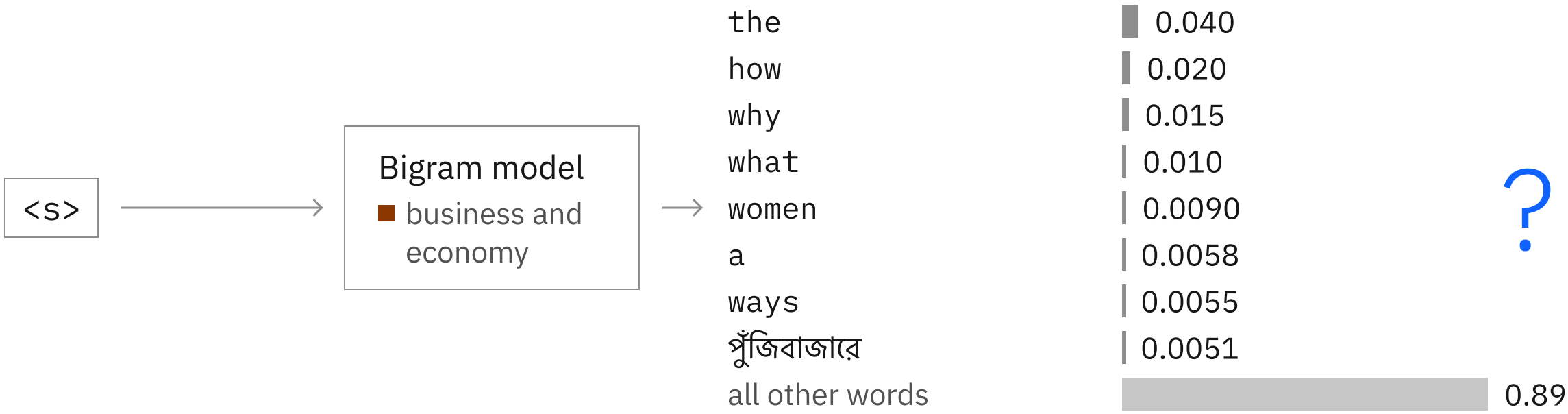
Loss in classification



$$\text{loss} = -\log 0.95 = 0.05$$

On the dev set, the loss is the mean over the 4,000 articles.

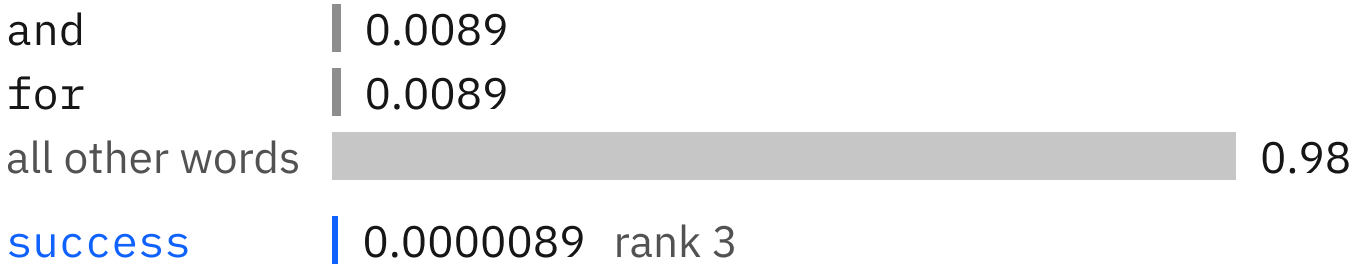
Loss in generation



To compute a loss, we need the true class. What is it here?

The true next word

<s> what you don t know about **overnight** success



Word	p	$-\log p$
what	0.010	4.61
you	0.061	2.80
don	0.022	3.80
t	0.60	0.50
know	0.032	3.46
about	0.086	2.46
overnight	0.0000019	13.17
success	0.0000089	11.63
sum		**42.43**

Classification gives one loss per article. Generation gives one loss per word.

The loss of a text

```
<s> what you don t know about overnight success
```

$$p(\text{headline}) = p(\text{what} \mid <\text{s}>) \times p(\text{you} \mid \text{what}) \times \cdots \times p(\text{success} \mid \text{overnight})$$

$$-\log p(\text{headline}) = -\log p(\text{what} \mid <\text{s}>) - \log p(\text{you} \mid \text{what}) - \cdots - \log p(\text{success} \mid \text{overnight})$$

$$= 4.61 + 2.80 + 3.80 + 0.50 + 3.46 + 2.46 + 13.17 + 11.63 = \mathbf{42.43}$$

So the loss of the headline is the sum of the losses of its words.

Mean loss per word

```
<s> what you don t know about overnight success
```

$$\frac{42.43}{8} = 5.30$$

$$\text{loss} = -\frac{1}{n}\sum_{i=1}^{n} \log p(w_i \mid \text{previous words})$$

	Classification	Generation
What is predicted	the topic	the next word
Classes	6 topics	110,609 words
Losses per article	1	one per word
The mean is over	4,000 dev articles	174,640 dev words

The loss is the same in both; only the classes and the number of losses change.

Perplexity



$$\text{loss} = -\log(1/3) = \log 3 = 1.10$$

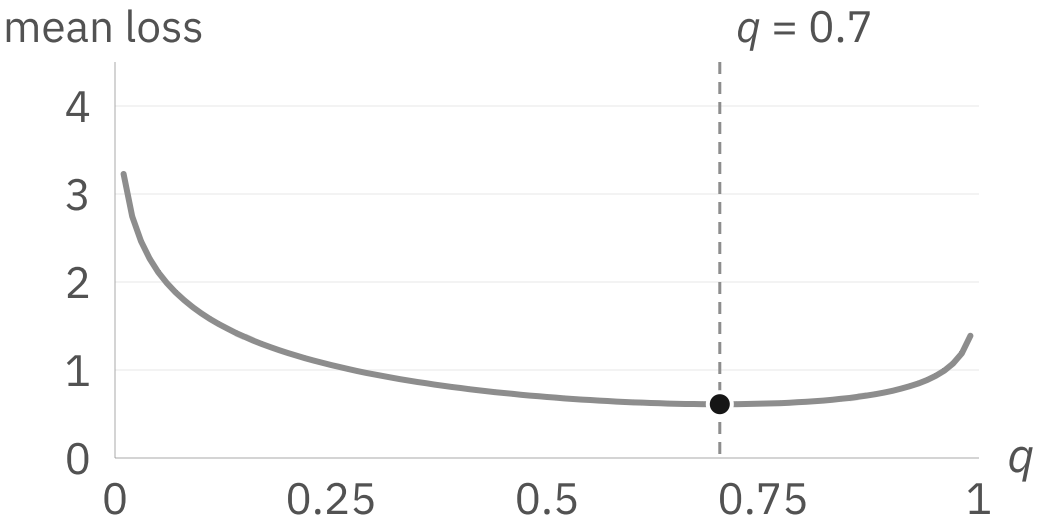
$$\text{perplexity} = e^{\text{loss}} = e^{\log 3} = 3$$



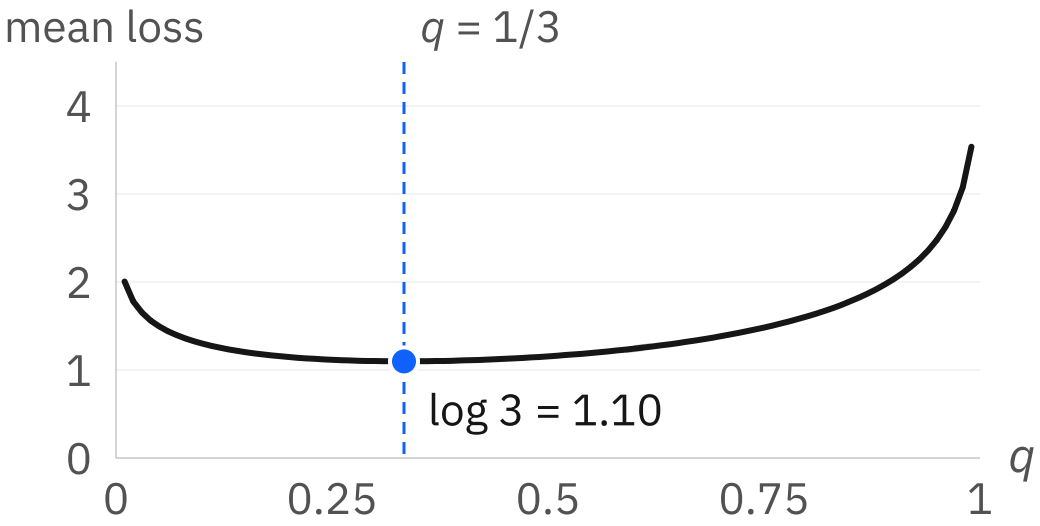
Perplexity depends on the test text, so we compare models on the same text.

The lowest possible loss

Shares: ■ sports 0.7, ■ entertainment 0.3



Shares: `wins`, `star`, `award`, 1/3 each



$$-\sum_{w} p(w) \log q(w) \;\geq\; -\sum_{w} p(w) \log p(w)$$

$p(w)$: the share of w in the text

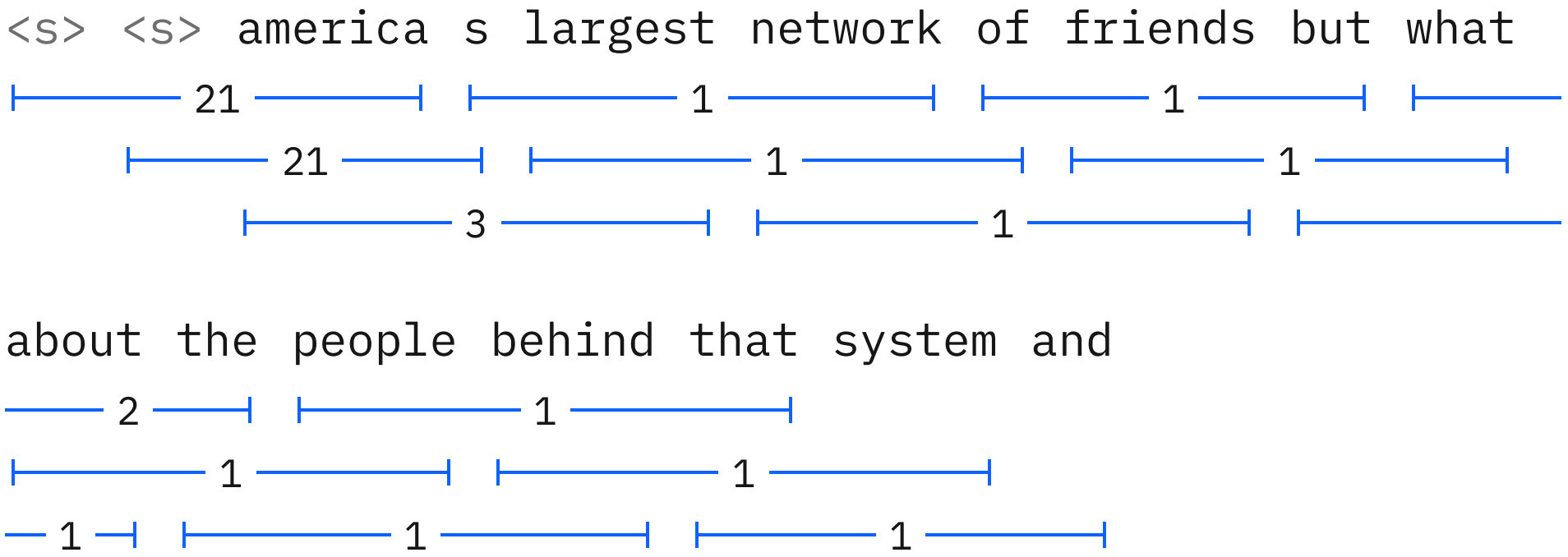
$q(w)$: the probability the model gives to w

Counting gives the shares in the training texts. Do they also fit new text?

Trigrams in generation

- Trigram model of business and economy, nothing added to the counts

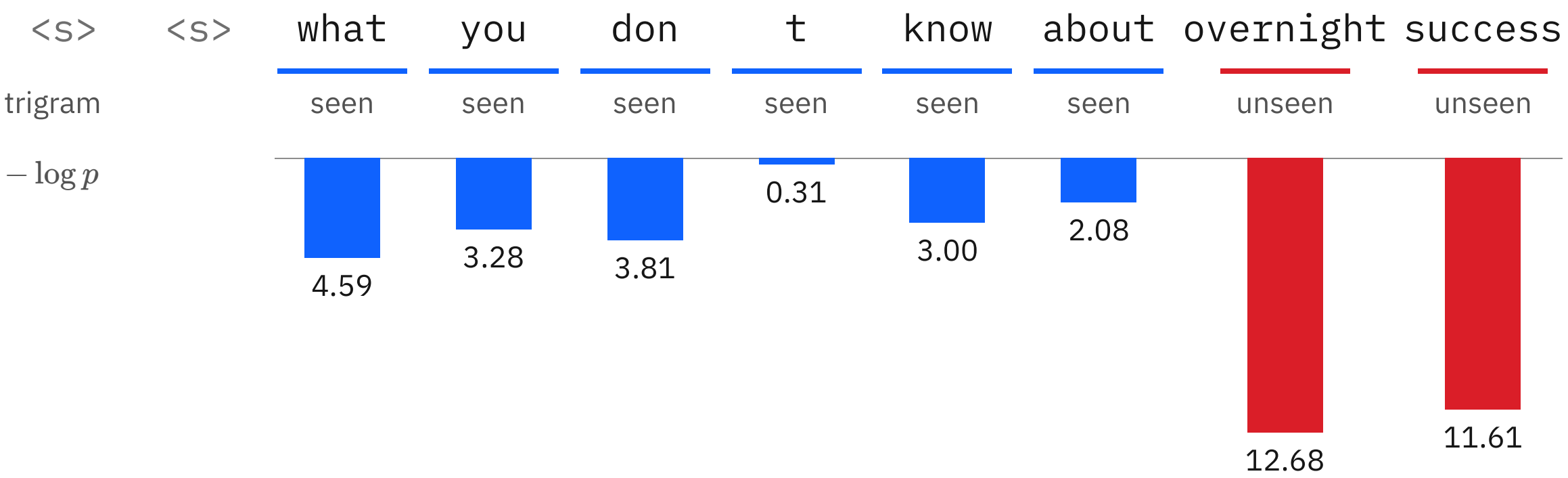
Count in its training texts



So the generated text repeats parts of the training texts.

Trigrams in new text

- Trigram model of business and economy



Without that small number, its probability would be 0 and the loss would be infinite.

Why is the trigram perplexity high?

	Words with an unseen n-gram of the 174,640 dev words	Mean loss per word		
		seen	unseen	all
Unigram	3.5%	8.36	14.96	8.59
Bigram	53.4%	4.28	12.26	8.54
Trigram	84.3%	2.77	11.85	10.43

Problem: the trigram model gives most words of new text a very small probability.

Part 5: Unseen n-grams

Every n-gram needs a probability above 0.

A new headline

Test headline `Star film wins award`

`<s> star` | `star film` | `film wins` | `wins award` | `award <s>`

- sports

	<s>	team	wins	match	star	award	film
<s>	0	2/3	0	0	1/3	0	0
team	0	0	1/2	0	1/2	0	0
wins	0	0	0	2/3	0	1/3	0
match	2/2	0	0	0	0	0	0
star	0	0	2/2	0	0	0	**0**
award	1/1	0	0	0	0	0	0
film	–	–	–	–	–	–	–

- entertainment

	<s>	team	wins	match	star	award	film
<s>	0	0	0	0	1/3	0	2/3
team	–	–	–	–	–	–	–
wins	0	0	0	0	0	2/3	1/3
match	–	–	–	–	–	–	–
star	0	0	2/2	0	0	0	**0**
award	3/3	0	0	0	0	0	0
film	0	0	1/3	0	1/3	1/3	0

- $p(\text{text} \mid \text{entertainment}) = 1/3 \times 0 \times 1/3 \times 2/3 \times 3/3 = 0$ Loss $-\log 0 = \infty$
- $p(\text{text} \mid \text{sports}) = 1/3 \times 0 \times \cdots = 0$

Problem: a single pair that never occurs in training makes the probability of the whole text 0.

Add-one smoothing

- entertainment

	<s>	team	wins	match	star	award	film	**Total**
	+1	**+1**	**+1**	**+1**	**+1**	**+1**	**+1**	**+7**
star	1	1	3	1	1	1	1	**9**
$p(w \mid \text{star})$	1/9	1/9	3/9	1/9	1/9	1/9	1/9	**1**

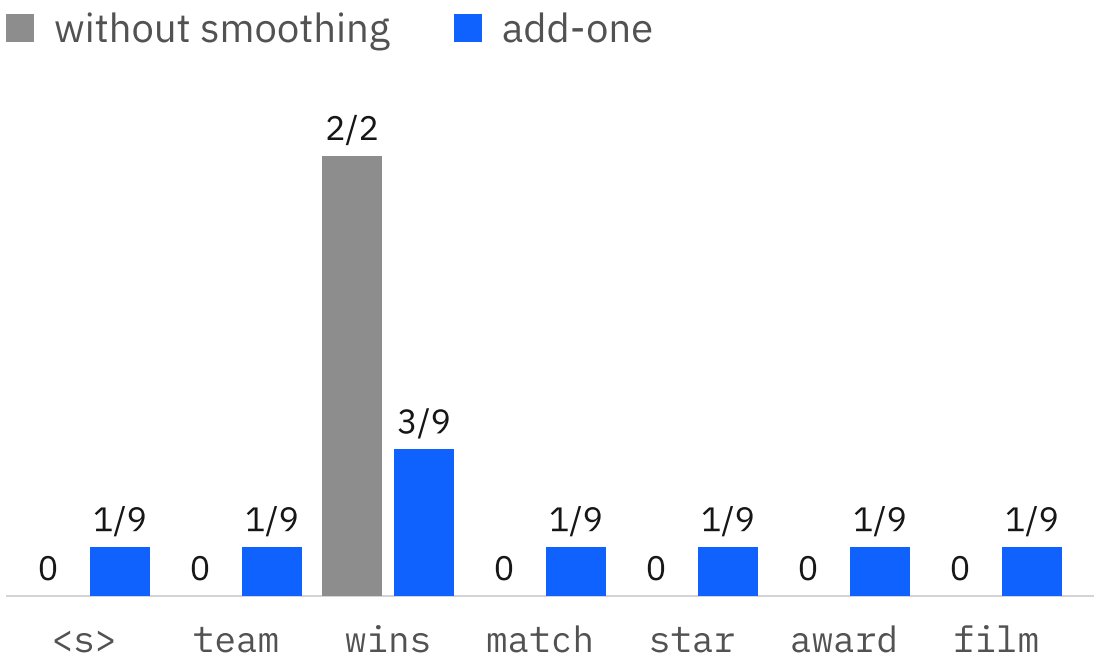
$$p(w \mid v) = \frac{\text{count}(v\,w) + 1}{\text{count}(v) + V}$$

$$p(\text{film} \mid \text{star}) = \frac{0+1}{2+7} = \frac{1}{9}$$

Solved: $p(\text{film} \mid \text{star}) = 1/9$, so the headline Star film wins award gets a probability above 0.

The cost of add-one

$p(w \mid \text{star})$ in entertainment



Dev loss per word with add-one, lower is better



Problem: in C1, add-one adds $V = 110{,}609$ to every row total, so it gives too much probability to pairs that never occur.

What does adding 1 do?

The row of `presidential`, counted in all 40,000 training texts

185 real counts +1 for each of the 110,609 next words



110,794

Without smoothing $$p(\text{election} \mid \text{presidential}) = \frac{31}{185} = 0.17$$

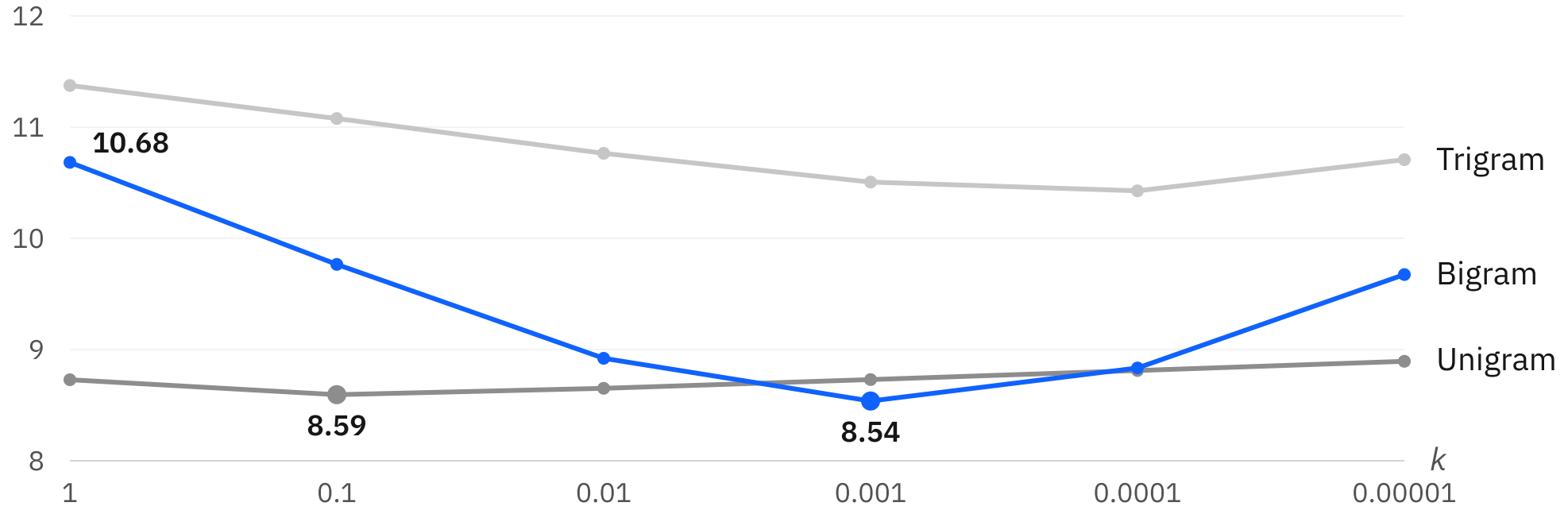
With add-one $$p(\text{election} \mid \text{presidential}) = \frac{31+1}{185+110{,}609} = 0.00029$$ **580 times smaller**

Problem: words never seen after presidential now get 99.8% of the probability.

Add-k smoothing

$$p(w \mid v) = \frac{\text{count}(v\,w) + k}{\text{count}(v) + kV}$$

Dev loss per word, lower is better



Solved: with $k = 0.001$, the bigram model predicts the dev text better than the unigram model.

Backoff

> The vote to leave the EU has sparked a panic among investors around the world.

		Count in the training texts
Trigram	panic among investors	0
Bigram	among investors	0
Unigram	**Add-one** 1/110,609 = 0.0000090 for every next word **Backoff** 64/1,811,637 = 0.000035 for investors	investors 64

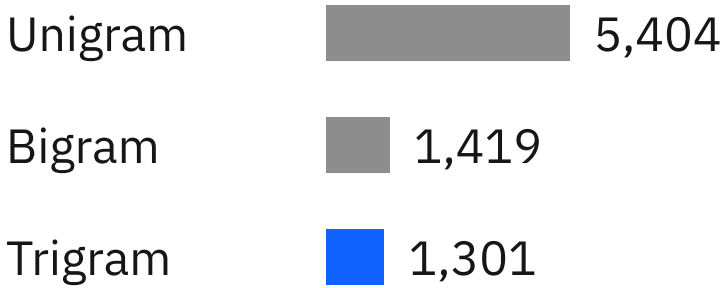
Add-one gives every word after panic among the same 1/110,609. Backoff gives investors 64/1,811,637, from its own count.

Perplexity with interpolation

$$\underbrace{\lambda_3\, p_3}_{\text{trigram}} + \underbrace{\lambda_2\, p_2}_{\text{bigram}} + \underbrace{\lambda_1\, p_1}_{\text{unigram}}$$

The weights add up to 1.

Perplexity on the dev articles, with interpolation



Solved: the trigram model now has the lowest perplexity.

Handling unseen n-grams

Method	What it does	Its weakness
Add-one	Adds 1 to every count	Gives far too much to unseen n-grams: 99.8% of the row after presidential
Add-k	Adds a small k, chosen on the dev set	One k for every row. Bigram dev loss: 10.68 at k = 1, 8.54 at k = 0.001
Backoff	Uses the longest n-gram that was seen	Drops context. To add up to 1, the longer n-grams must give up some probability
Interpolation	Mixes the trigram, bigram and unigram probabilities	One set of weights for every history

All four only repair counts. None of them knows that two words are alike.

What we learned

Word order	The chain rule conditions each word on every word before it. The Markov assumption keeps only the last N – 1.	$p(w_i \mid w_1 \dots w_{i-1}) \approx p(w_i \mid w_{i-N+1} \dots w_{i-1})$
The bigram model	Each topic has a table of pairs, counted from its training texts.	$p(w \mid v, k) = \frac{\text{count}(v\,w, k)}{\text{count}(v, k)}$
More context	Longer histories are rarer, so the trigram model classifies worst.	Dev macro-F1: unigram 0.836, bigram 0.820, trigram 0.733
Generation	Generation chooses the next word. A model is measured by its perplexity on new text: e to the power of the mean loss per word.	Dev perplexity at the best k: unigram 5,404, bigram 5,103, trigram 33,752
Unseen n-grams	Smoothing gives every n-gram a probability above 0. Backoff and interpolation also use shorter histories.	Dev perplexity with interpolation: unigram 5,404, bigram 1,419, trigram 1,301

References

Textbook

Jurafsky, D. and Martin, J. H. (2026). *Speech and Language Processing*, 3rd ed. draft, August 2026. Chapter 3, sections 3.1–3.7, pp. 68–88 (n-gram language models: the chain rule, eq. 3.4; the Markov assumption, eqs. 3.7 and 3.8; probabilities from counts, eq. 3.11; perplexity, section 3.3, p. 76, eqs. 3.14 to 3.16; Laplace smoothing, eq. 3.26; add-k smoothing, eq. 3.28; backoff, section 3.6.4; linear interpolation with weights set on a held-out corpus, section 3.6.3, eq. 3.29, p. 84; the cross-entropy as an upper bound on the entropy, eq. 3.41, p. 87).

Data and code

The script c03_build.py (the class 1 tokenizer; the unigram, bigram and trigram models; add-k smoothing; interpolation), on the C1 training set (40,000 texts) and dev set (4,000 texts).

Thank you.